Jetson Xavier NX Board Setup Manual

RC Car Assembly Manual Series No.3

This manual is the third one of our RC car assembly manual series. In this manual, we will explain the Jetson Xavier NX board setup processes including Ubuntu, Nvidia jetpack and pytorch installation. Please read and follow the instructions carefully. There may be changes in parts due to future discontinuation issues.

RC Car Assembly Manual Series

1. RC Car Hardware Assembly Manual
2. VESC Setup Manual
3. Jetson Xavier NX Setup Manual
4. Keyboard Control with ROS2
5. RC Car Calibration Manual

RLLAB

http://rllab.snu.ac.kr

Last modified: 25.09.22

Table of contents

0. Before Starting (Rebooting Jetson)

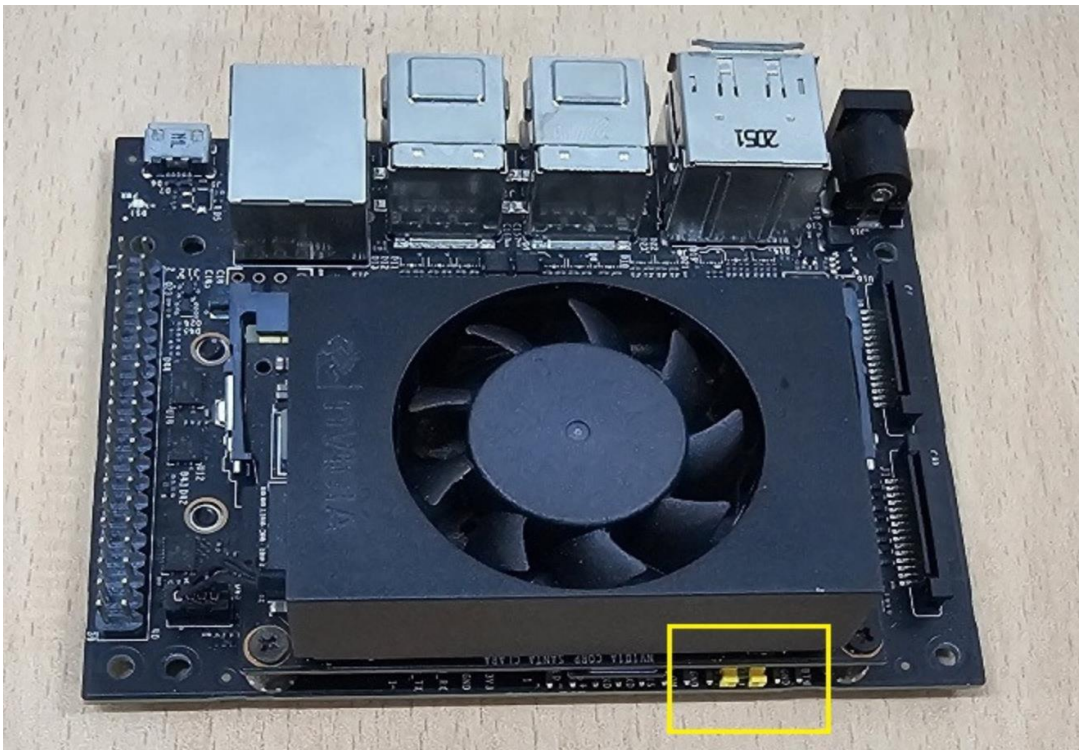
1. Install Ubuntu 20.04 on Jetson Xavier NX (with eMMC + SSD)

2. Install full Nvidia jetpack for Jetson board
   2.1. Mount SSD
   2.2. Copy the entire ubuntu system from eMMC to SSD
   2.3. Install whole Nvidia jetpack for Jetson

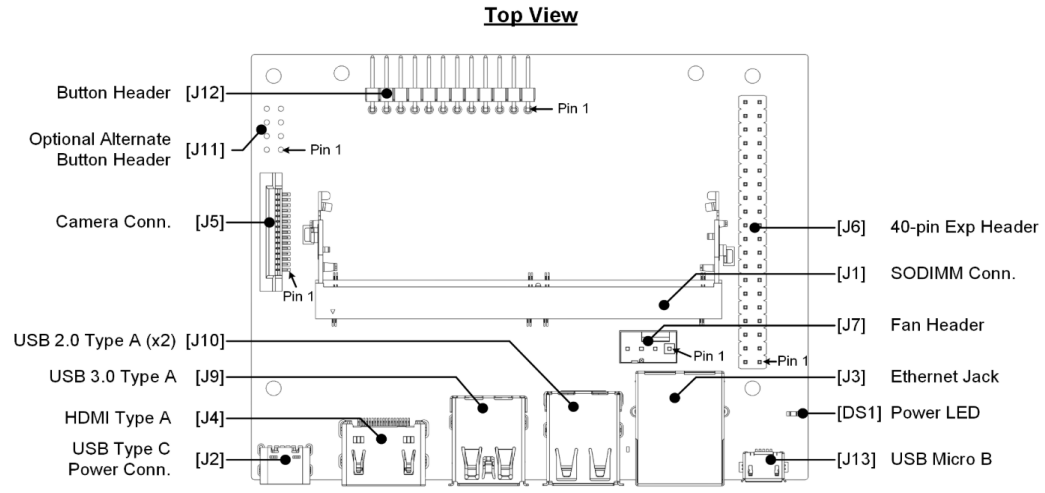
3. Install virtual environment
   3.1. Install virtualenv
   3.2. Install pytorch on Jetson

0. Rebooting Jetson

0-0. To operate in recovery mode, you will need a jumper pin and a Micro-B type USB cable. As shown in the diagram below, attach the jumper pin to pins 9 and 10 of the 12-pin button pin header. For details on the 12-pin layout, see the picture below.

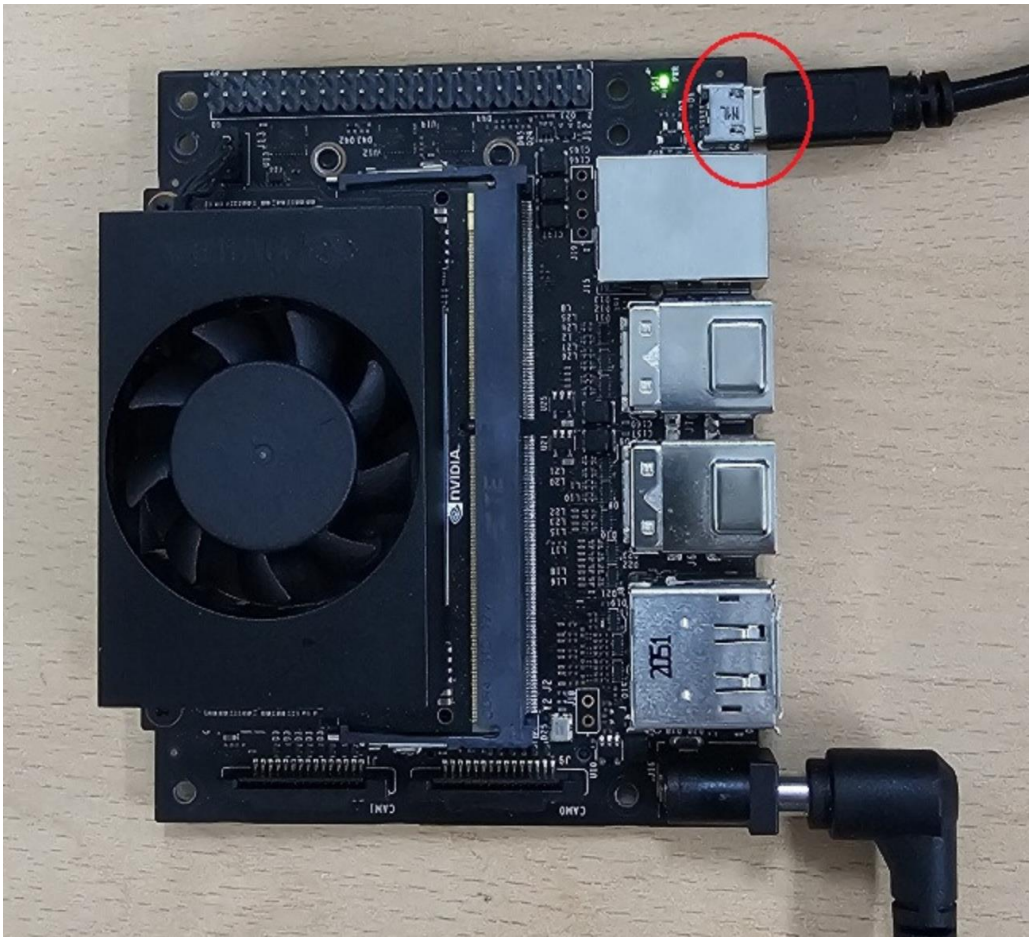


Carrier Board Layout



Jetson Nano 2GB Top View

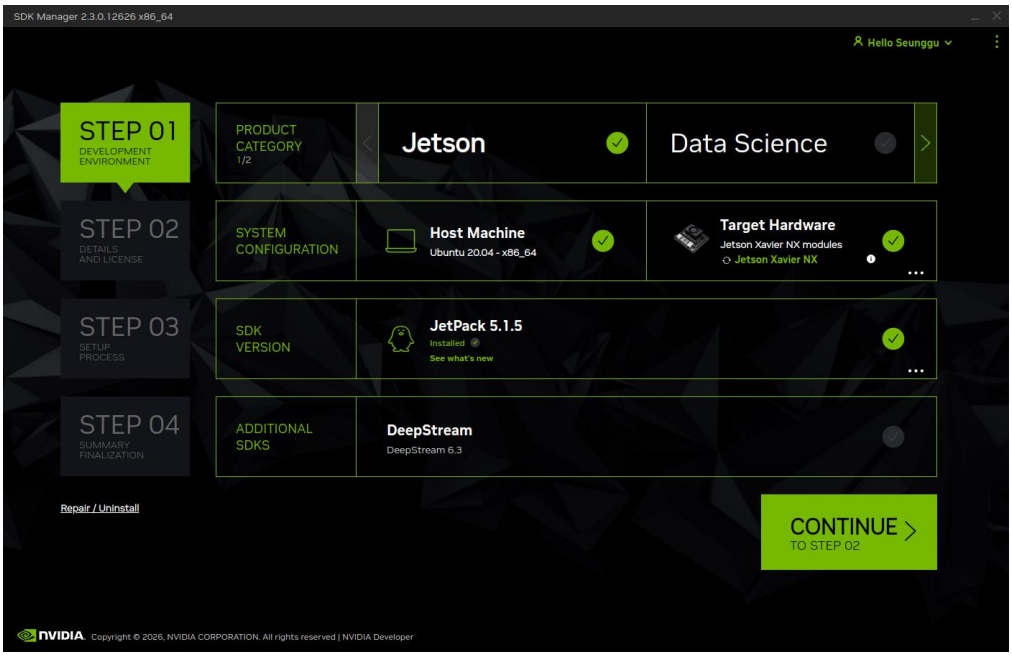
0-1. Connect the Micro USB cable as shown in the diagram below and link it to the host computer. After plugging in the power, remove the jumper pin and move to 1-1.



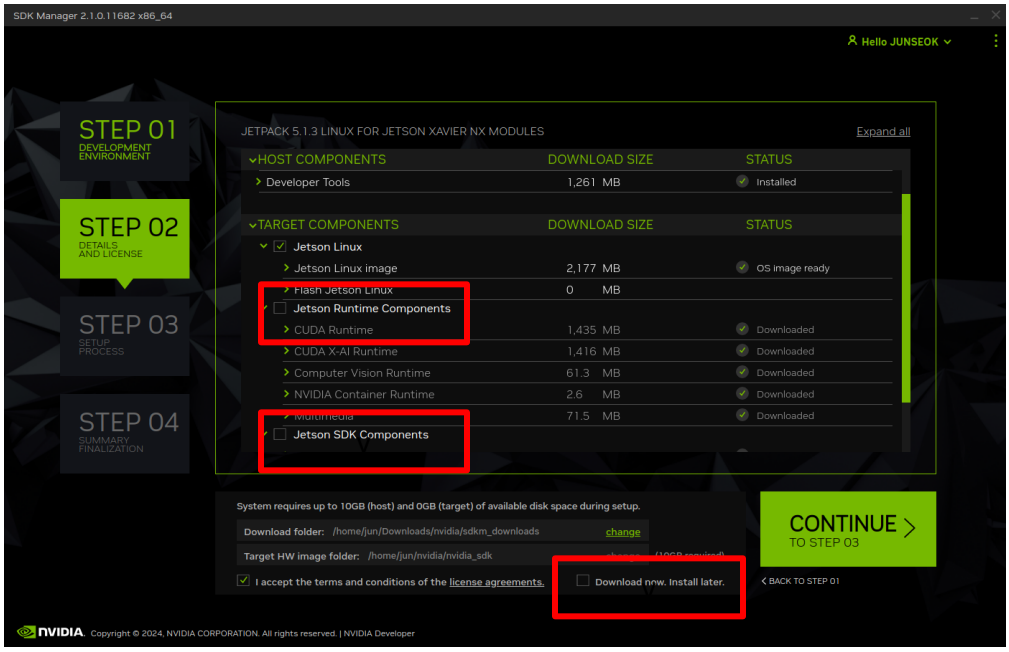
1. Install Ubuntu 20.04 on Jetson Xavier NX (with eMMC + SSD)

1-0. For Nvidia Jetson Xavier NX boards produced in China, Ubuntu 18.04 is pre-installed on the 8GB eMMC, and the installation process for Ubuntu 18.04 begins at boot. The installation follows the same steps as those outlined in the dual-boot manual. Please remember to note your username and password during this process.

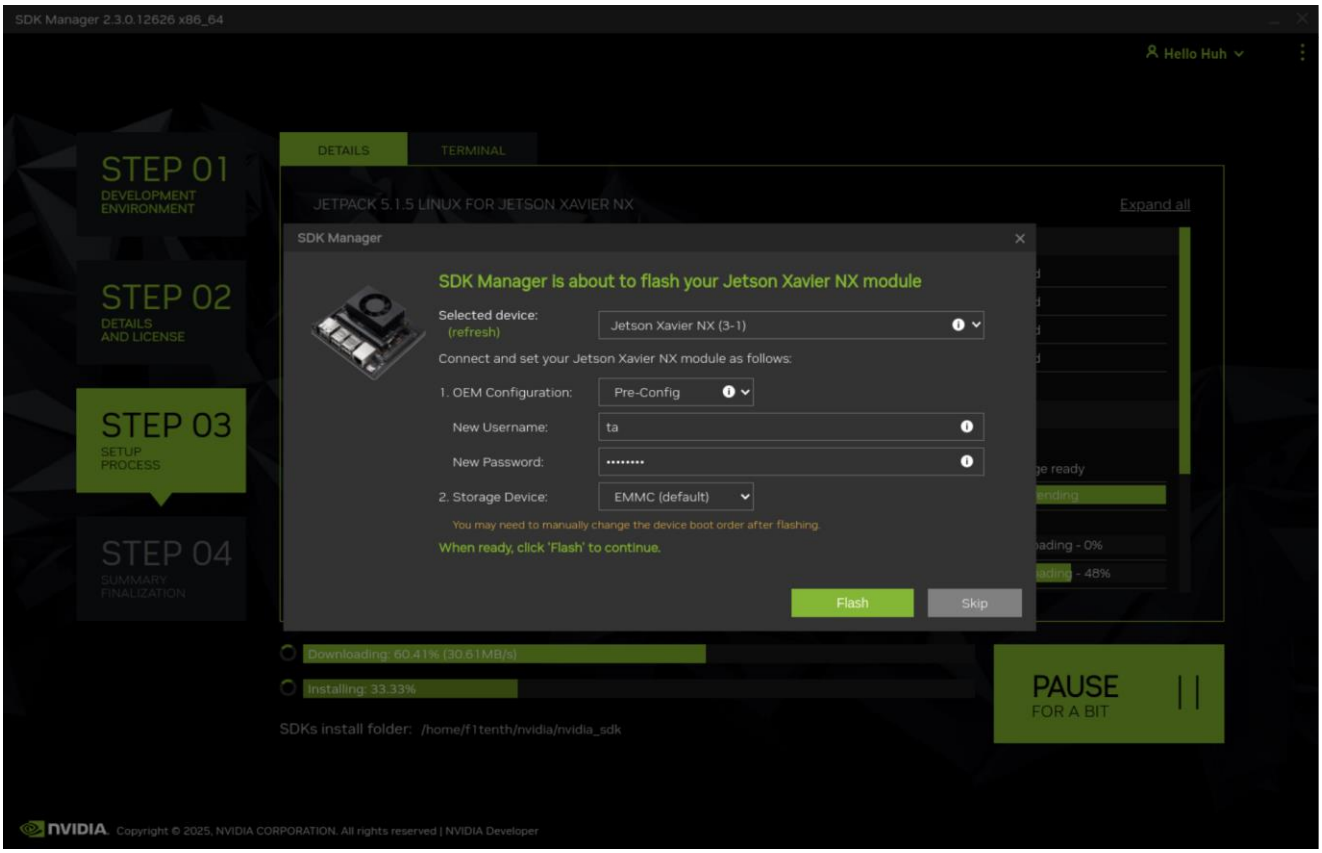
1-1. Please check if the power and micro-5 pin of the Jetson board are connected to another computer where the Nvidia SDK Manager is installed. If the connection is successful, you can verify that the Jetson Xavier NX modules are successfully recognized in the “Target Hardware” section.



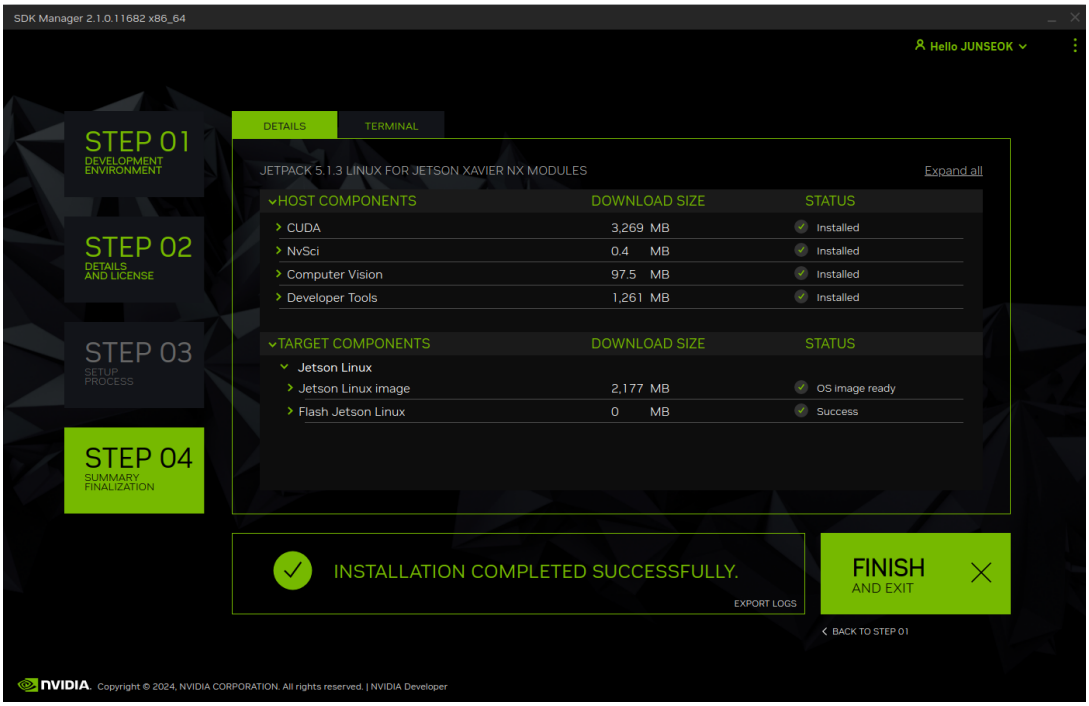
1-2. Uncheck the boxes for “Jetson Runtime Components,” “Jetson SDK Components,” and “Download now. Install later,” then press the “CONTINUE” button. This step is necessary because not unchecking these options can lead to errors due to the limited capacity of the eMMC.



1-3. During the installation process, a window will appear where you need to accurately enter the username and password that you will use in Ubuntu 20.04. After entering this information, press the “Flash” button to continue with the installation.



1-4. If the installation was successful, Ubuntu 20.04 has been installed. However, this does not mean that all the Jetpack components for the Jetson board have been installed. To complete the installation, please follow the subsequent steps.



2. Install full Nvidia jetpack for Jetson board

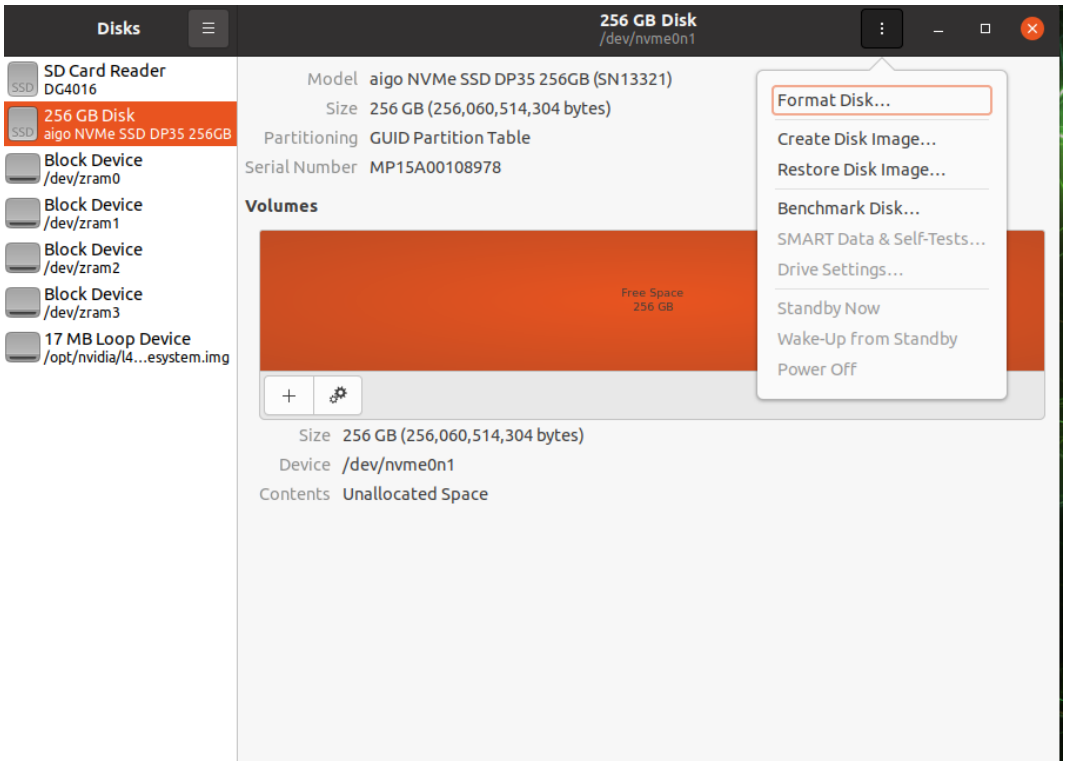
2-1. Format SSD

When turning on the Jetson board, you can find the board is upgraded to 20.04. There will be 128GB or 256GB SSD attached on the jetson board, so we are going to install all Ubuntu system and following packages in the SSD.

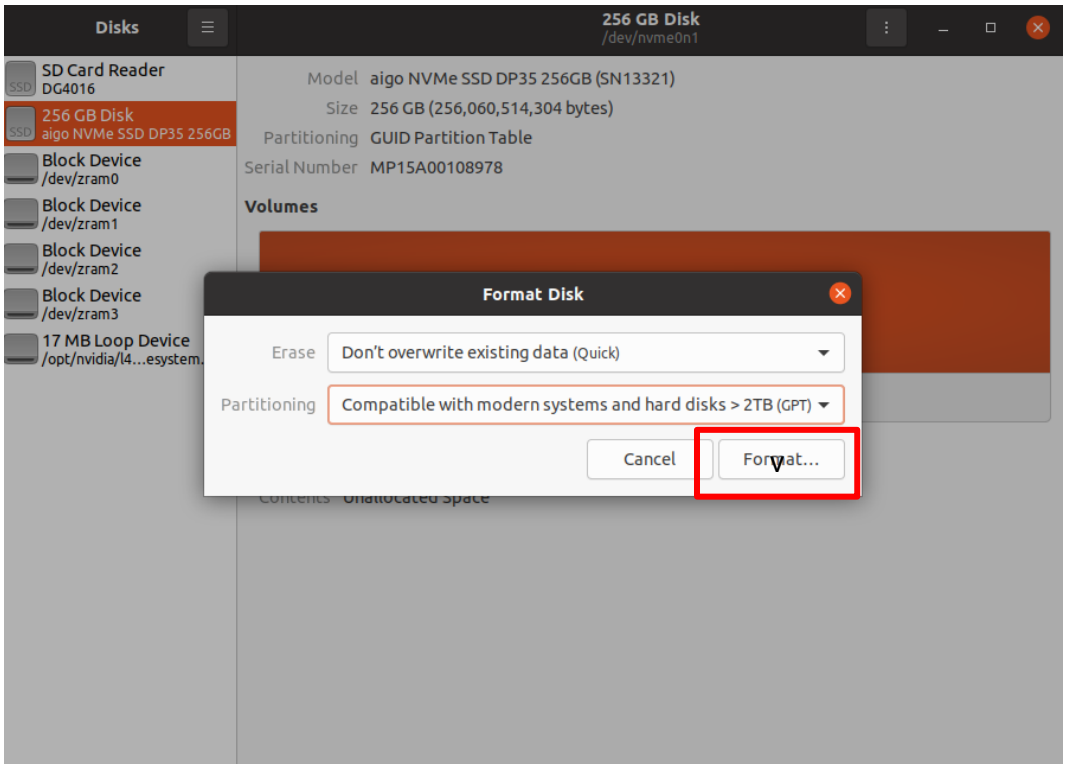
Search “disks” in system menu.

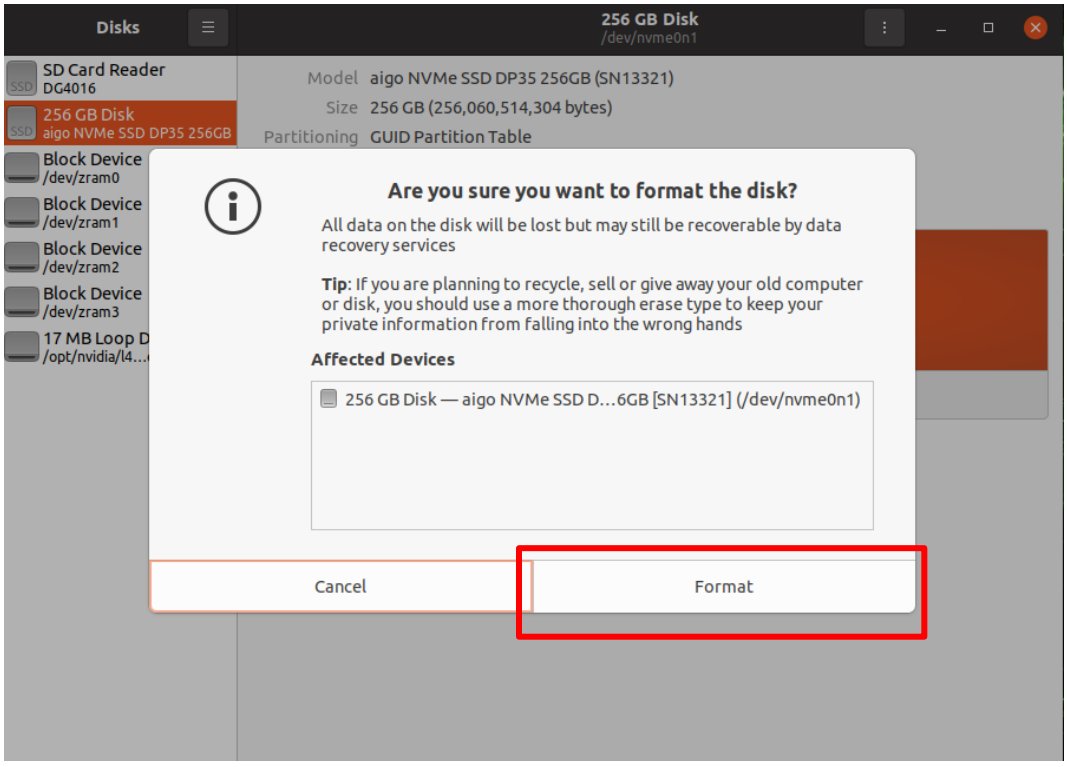


You can find 256 GB Disk in the left side of the “Disks” window. Select it, and click three-dot menu button on the top. Select “Format Disk”.



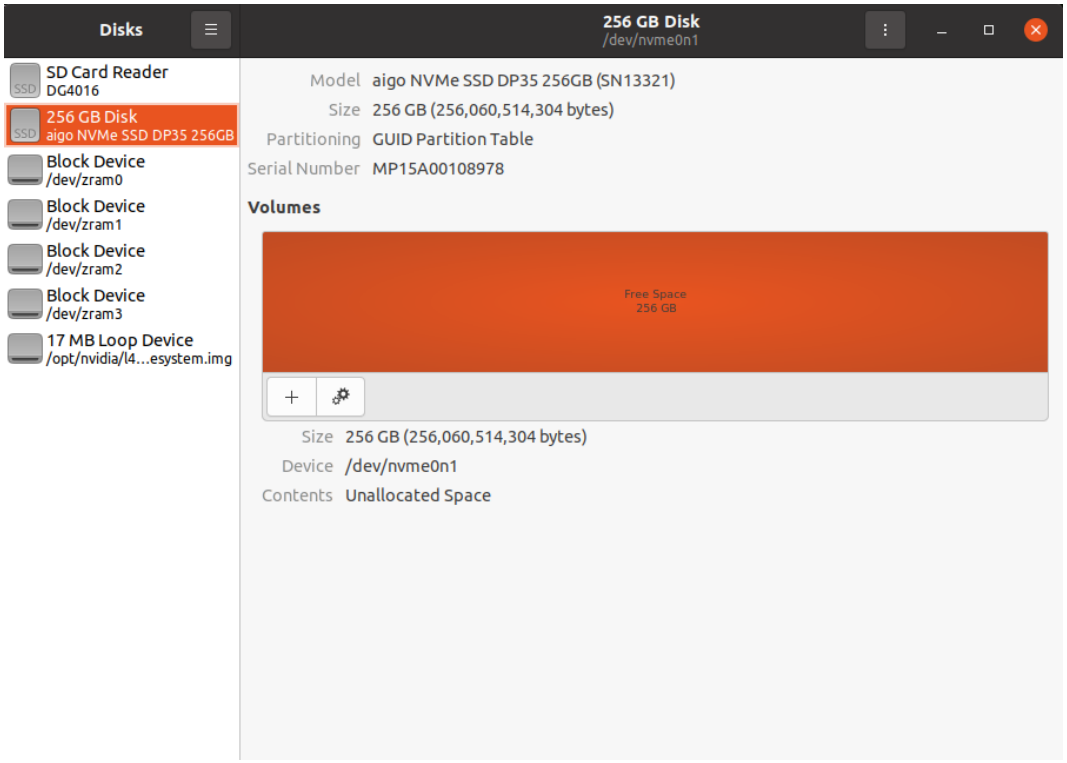
Follow the steps in the figures.



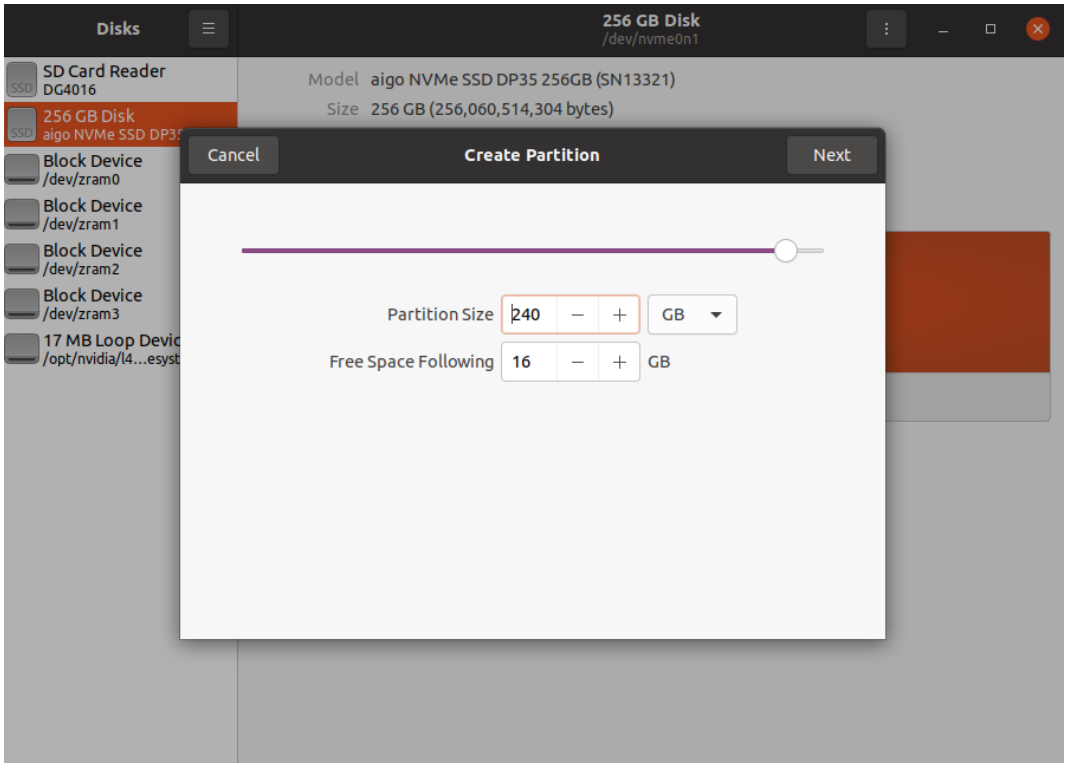


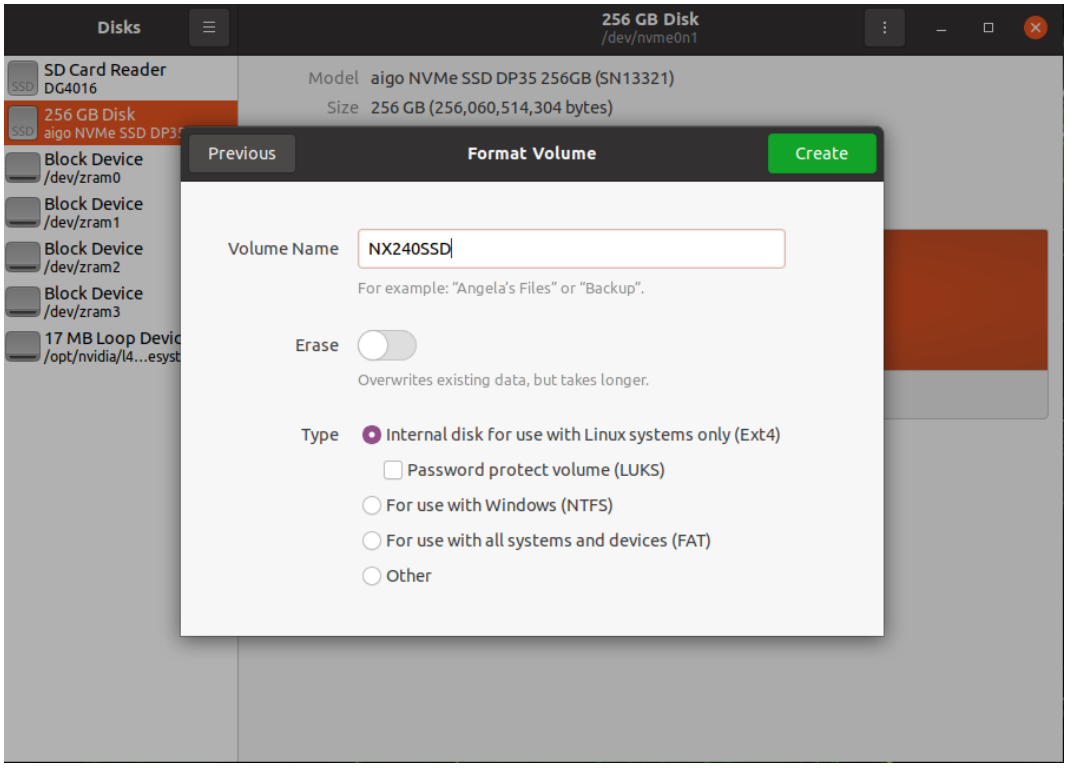
2-2. Make a new partition in the SSD

To mount the SSD to the Ubuntu system, we need to make a partition on it. Click “+” button after selecting 256 GB Disk.

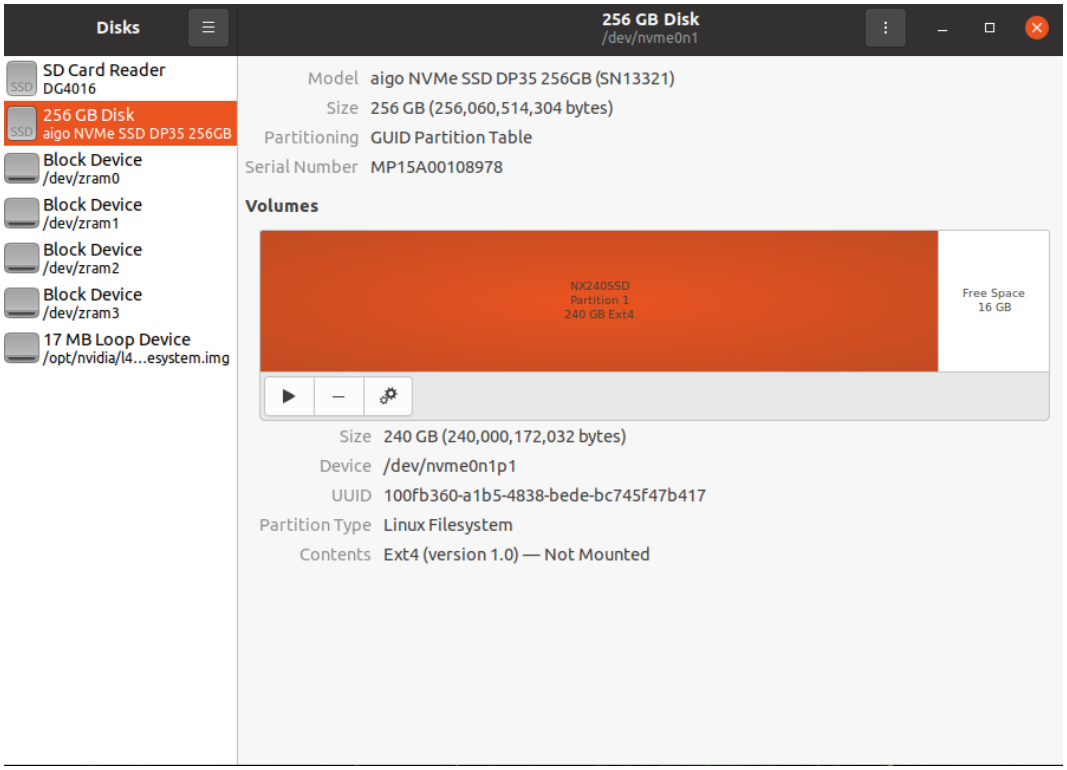


Adjust Partition size to have Free space about 16GB, then click Next.





You now can see a new partition has been created.



2-3. Copy the entire ubuntu system from eMMC to SSD

First, we need some codes to copy the system and bootloader from eMMC to SSD. Download the codes from
https://github.com/jetsonhacks/rootOnNVMe.
Make sure that the system is connected to the internet.

```
git clone https://github.com/jetsonhacks/rootOnNVMe
```

After downloading is finished, navigate to rootOnNVMe folder,

```
cd rootOnNVMe
```

and run .sh files.

```
./copy-rootfs-ssd.sh
```

```
ta@ubuntu:~/rootOnNVMe$ ls
copy-rootfs-ssd.sh  data  LICENSE  README.md  setup-service.sh
ta@ubuntu:~/rootOnNVMe$ ./copy-rootfs-ssd.sh
  1,042,893,572  98%   72.10MB/s    0:00:13 (xfr#9116, ir-chk=1091/12055)
```

```
./setup-service.sh
```

```
ta@ubuntu:~/rootOnNVMe$ ./setup-service.sh
Created symlink /etc/systemd/system/default.target.wants/setssdroot.service → /etc/systemd/system/setssdroot.service.
Service to set the rootfs to the SSD installed.
Make sure that you have copied the rootfs to SSD.
Reboot for changes to take effect.
```

After the process, reboot your machine.

```
sudo reboot
```

The boot priority will already be set to SSD over eMMC, so the machine automatically boot with ubuntu image in SSD.
After reboot, check if you are now really on the system in SSD.

```
ta@ubuntu:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
none            3.3G     0  3.3G   0% /dev
tmpfs           3.4G   36K  3.4G   1% /dev/shm
tmpfs           684M   35M  649M   6% /run
tmpfs           5.0M  4.0K  5.0M   1% /run/lock
tmpfs           3.4G     0  3.4G   0% /sys/fs/cgroup
/dev/nvme0n1p1  234G  6.0G  216G   3% /
tmpfs           684M   16K  684M   1% /run/user/124
tmpfs           684M   20K  684M   1% /run/user/1000
```

2-4. Install whole Nvidia jetpack for Jetson board

Install the whole Nvidia jetpack for Jetson board.

```
sudo apt-get update
```

```
sudo apt install nvidia-jetpack
```

```
ta@ubuntu:~$ sudo apt install nvidia-jetpack
[sudo] password for ta:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  apt-clone archdetect-deb bogl-bterm busybox-static cryptsetup-bin
  dctrl-tools dpkg-repack gdal-data gir1.2-timezonemap-1.0 gir1.2-xkl-1.0
  grub-common libaec0 libarmadillo9 libarpack2 libavcodec-dev libavformat-dev
  libavresample-dev libavresample4 libavutil-dev libcfitsio8 libcharls2
  libdap25 libdapclient6v5 libdc1394-22-dev libdebian-installer4 libepsilon1
  libevent-core-2.1-7 libevent-pthreads-2.1-7 libexif-dev libfreexl1 libfyba0
```

We need to make it possible for the system to recognize CUDA program we installed.

Open the ~/.bashrc file with vim. ~/.bashrc file is a file that allows you to change the default settings for bash, which is set as the default terminal in Ubuntu.

```
gedit ~/.bashrc
```

Add the following lines at the end of the ~/.bashrc file.

```
export PATH=$PATH:/usr/local/cuda/bin
export CUDAIR=/usr/local/cuda
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/cuda/lib64
```

Save and exit.

Apply the changes in the ~/.bashrc file in current terminal.

```
source ~/.bashrc
```

Check your CUDA successfully downloaded and detected by the system.

```
nvcc -V
```

You can see the version of CUDA you installed.

```
ta@ubuntu:~$ nvcc -V
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2022 NVIDIA Corporation
Built on Sun_Oct_23_22:16:07_PDT_2022
Cuda compilation tools, release 11.4, V11.4.315
Build cuda_11.4.r11.4/compiler.31964100_0
```

3. Install pytorch for Jetson

3-1. Install virtualenv

A Python virtual environment is an isolated environment that allows you to manage dependencies for different projects independently. It creates a separate directory that contains a specific Python interpreter and its own set of installed packages, ensuring that dependencies from one project don't interfere with those of another. This is particularly useful when working on multiple projects that require different versions of the same package or when you want to avoid affecting the global Python environment on your system. By using virtual environments, you can maintain clean, organized, and conflict-free project setups.

There are several programs to make use of python virtual environment such as venv, virtualenv and Anaconda. We'll use virtualenv for our project.

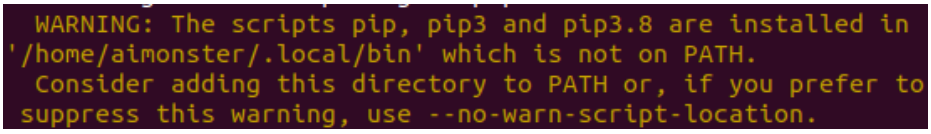
** Caution: We cannot install Anaconda, a famous virtual environment program, on Jetson since it is based on arm-based architecture.*

Install required programs.

```
sudo apt-get install python3-pip libopenblas-base libopenmpi-dev libomp-dev
```

```
python3 -m pip install --upgrade pip
```

If warning in picture below shows up,



open your bashrc file,

```
gedit ~/.bashrc
```

and add the following line at the end of the ~/.bashrc file. Modifiy '*user_name*' to your own name.

```
export PATH=$PATH:/home/{user_name}/.local/bin
```

Apply the changes in the ~/.bashrc file in current terminal.

```
source ~/.bashrc
```

Next, install virtualenv.

```
python3 -m pip install virtualenv
```

Check the version.

```
virtualenv --version
```

You can make a new virtual environment with command

```
virtualenv PATH/ENV_NAME --python=PYTHON_VERSION
```

For example, if you want to make a virtualenv with name “torch210” in the folder named “venv” in your /home directory, the command should be

```
virtualenv venv/torch210 --python=python3.8
```

You can activate the virtualenv you made with command

```
source PATH/ENV_NAME/bin/activate
```

For instance, with the ‘torch210’ environment we made above with command

```
source venv/torch210/bin/activate
```

And you can see (torch210) in front of the current path in the terminal.

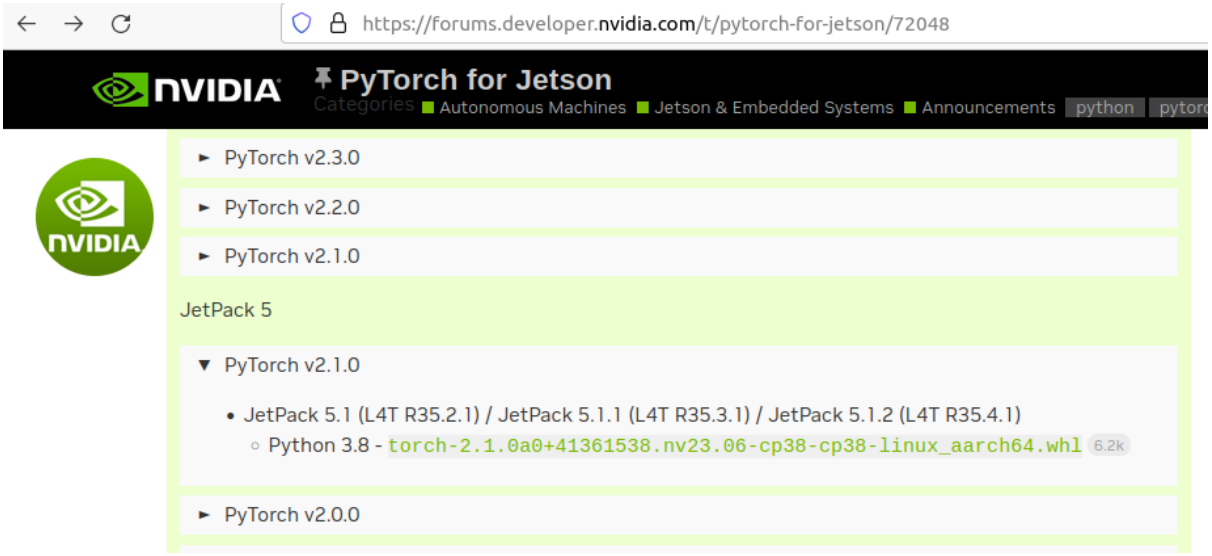
Deactivation of the environment can be done with

```
deactivate
```

3-2. Install pytorch on Jetson

From now on, every Python package you install with pip should be installed within your virtual environment after activating it. Also, always make sure that your virtual environment is activated before running your program. When installing with 'sudo apt', it doesn't matter because the program is being installed on the computer, not within the Python environment.

Download wheel from https://forums.developer.nvidia.com/t/pytorch-for-jetson/72048.
We have installed jetpack 5.1.3 in section 1, so download PyTorch v2.1.0 whl file by clicking the link in the page shown in the below picture.
(Since it's been confirmed that the 'wget' command has a bug, please download the file by clicking the link instead.)




```
(torch210) pip install 'Cython<3'
```

Install pytorch with the whl file downloaded before.

```
(torch210) pip install torch-2.1.0a0+41361538.nv23.06-cp38-cp38-linux_aarch64.whl
```

```
(torch210) ta@ubuntu:~/Downloads$ pip install torch-2.1.0a0+41361538.nv23.06-cp38-cp38-linux_aarch64.whl
Processing ./torch-2.1.0a0+41361538.nv23.06-cp38-cp38-linux_aarch64.whl
Collecting fsspec
  Downloading fsspec-2024.6.0-py3-none-any.whl (176 kB)
     |████████████████████████████████| 176 kB 4.1 MB/s
Collecting networkx
  Downloading networkx-3.1-py3-none-any.whl (2.1 MB)
     |████████████████████████████████| 2.1 MB 11.0 MB/s
Collecting filelock
  Downloading filelock-3.15.3-py3-none-any.whl (16 kB)
```